By TOUCH DEN · November 11, 2026

The Boot Process

Video 1 · From Power Button to Running OS

How Computers Work · Section 9

In this video, you will learn:

- The boot chain, power-on to desktop
- Firmware (BIOS/UEFI) and the bootloader
- Loading the kernel, and init

Boot is a chain

Press power: RAM is empty, the OS is just a file on disk. Seconds later: a running desktop. That journey is the **boot process**.

A **chain**, each stage finds, checks, and starts the next:

1. **Firmware** wakes the hardware, finds a boot drive.
2. **Bootloader** finds and loads the kernel.
3. **Kernel** takes over, sets up the system.
4. **Init** starts the first programs → login → desktop.

> **Whoever controls an early link controls everything after it.**

Firmware: BIOS and UEFI

The first code isn't on the disk, it's **firmware**: a permanent program on a **motherboard chip**, running the instant power arrives.

Its job: wake and test the hardware, find a bootable drive, run the first software on it (the bootloader).

- **BIOS:** the old standard. **UEFI:** the modern replacement, can also **verify** what it boots.

Firmware runs **before the OS and antivirus**, so firmware malware survives OS reinstalls, even a new disk.

Bootloader, kernel, and init

- **Bootloader:** small program that finds the **kernel** on the drive, loads it, and starts it (may show a Windows/Linux menu). Runs **before** the kernel.
- **Kernel loads:** takes over, loads **drivers**, sets up **virtual memory** (lesson 6), starts the first process.
- **Init (PID 1):** the first user-space process, **ancestor of all others** (lesson 6's process tree). Leads to login and desktop.

firmware → bootloader → kernel → init → desktop.

Recap: the boot chain

- Boot is a **chain**: firmware → bootloader → kernel → init → desktop; each starts the next.
- **Firmware** (BIOS/UEFI) is on a chip, runs first, before the OS.
- **Init** is PID 1, the ancestor of every process.
- **Control an early link → control everything after.**

Next video: where malware hides at boot, and how Secure Boot defends it.

The Boot Process

Video 2 · Boot Malware and Secure Boot

How Computers Work · Section 9

In this video, you will learn:

- Why boot is a target: persistence
- Where malware hides (bootkits, rootkits)
- Secure Boot and the chain of trust

Where malware hides at boot

Malware wants to **survive restarts** (RAM is wiped on reboot), so it hooks into boot. **Earlier = more dangerous.**

Where	Name	Why dangerous
Firmware chip	Firmware implant	Before the OS; survives reinstall & new disk
Bootloader	Bootkit	Before the kernel; can load a tampered kernel
Kernel (driver)	Rootkit	Full privilege; **hides** files and processes
Startup programs	Autostart	Easier to find; runs after boot

Deep ones load **before** the tools that would detect them, then **lie** to them.

Why you can’t trust a compromised machine

A **rootkit** runs with full privilege and can hide its own files and processes from the OS’s own tools.

So the machine **lies about itself**.

> Serious forensics examines the disk from a **separate, trusted machine**, if the OS is compromised, you can’t trust what it reports.

The lesson-1 principle, control an early link and you hide from everything later, at full force.

Secure Boot: verify each link

If the danger is a tampered early link, the defense is to **check each link before trusting it**.

Secure Boot (in UEFI) uses **digital signatures** (course 09) to verify each stage is genuine before running it.

- A **chain of trust:** firmware runs only a signed bootloader → which loads only a signed kernel → …
- Tampered link → signature fails → boot **stops**.

Defense in depth, applied to the very **start** of the computer's life.

Recap: boot security

- Malware targets boot for **persistence**; **bootkits/rootkits** load early and **hide**.
- A compromised machine **can't be trusted to report on itself**, examine it from outside.
- **Secure Boot** verifies each stage's signature (**chain of trust**) to stop tampering early.

Next lesson: running many safe systems on one computer, virtual machines and containers (your future lab).